Ek 2 Yerel Container Uygulama Kılavuzu

Bulut Bilişim • Hafta 5 • Windows ve PowerShell

Bu kılavuz, hazır Ders Notları Kayıt Uygulaması için imaj oluşturmayı ve web uygulaması ile PostgreSQL veri servisini yerelde birlikte çalıştırmayı öğretir. Tamamlandığında tarayıcıdan kayıt ekleyebilir, container'ları yeniden oluşturduğunuzda verinin korunduğunu gösterebilir ve uygulamayı yeni bir imaj sürümüyle güncelleyebilirsiniz.

Kurulumun Ek 1 ile tamamlandığı varsayılır. Bulut hesabı, kredi kartı, Windows'a ayrı Python veya PostgreSQL kurulumu gerekmez. Docker Desktop Linux container modunda açık olmalıdır. Bulutta çalışma Ek 3 kapsamındadır; Kubernetes işlemleri bu kılavuzda yoktur.

1 Çalışma düzeni

Komutların tamamı normal PowerShell penceresinde çalıştırılır. Yönetici yetkisi veya Ubuntu terminaline geçiş gerekmez. Komut satırı istemini değil yalnız verilen komutu kopyalayın. Çok satırlı dosya içerikleri komut değildir; belirtilen dosyaya kaydedilir.

- Önce kaynak paketini açın ve çalışma klasörüne geçin.
- Gizli bilgi dosyalarını ve imaj sürümünü hazırlayın.
- Dockerfile ve .dockerignore ile uygulama imajını oluşturun.
- Compose dosyasıyla iki servisi çalıştırın; bağlantı ve veri kalıcılığını doğrulayın.
- Sürümü güncelleyin, kanıtları alın ve çalışma kaynaklarını sonlandırın.

Her adımın beklenen sonucunu kontrol etmeden sonraki adıma geçmeyin. Kaynak kodları hazırdır; container dosyalarını bu kılavuzdaki eksiksiz örneklerle siz oluşturursunuz. ZIP paketinin içine bu dosyalar önceden eklenmemiştir.

Her adımın kontrolü açıkça verilmiştir.

2 Kaynak paketini açma ve ortamı kontrol etme

Hafta-5-Uygulama.zip dosyasını ders kaynaklarından (BOYSIS üzerinden) indirin. Örnekte dosyanın İndirilenler klasöründe olduğu varsayılır; başka bir yere indirdiyseniz $zipPath değerini değiştirin. Klasörü yerel diskte tutun; eşitleme ve paylaşım sorunlarını azaltmak için OneDrive veya ağ klasörü kullanmayın.

```
$labRoot = Join-Path $env:USERPROFILE "bulut-dersi"
$zipPath = Join-Path $env:USERPROFILE "Downloads\Hafta-5-Uygulama.zip"
if (-not (Test-Path $zipPath)) { throw "ZIP dosyasi bulunamadi." }
New-Item -ItemType Directory -Path $labRoot -Force | Out-Null
Expand-Archive -LiteralPath $zipPath -DestinationPath $labRoot
Set-Location (Join-Path $labRoot "Hafta-5-Uygulama")
Get-Location
Get-ChildItem -Force
```

Beklenen sonuç: Çalışma klasöründe app.py ve requirements.txt; templates ve static klasörleri; README.md ve .env.example görünür. ZIP kendi içinde Hafta-5-Uygulama klasörünü içerir. Expand-Archive aynı yere tekrar açılırken mevcut dosyaların üstüne yazmayı istemeyin; güncel çalışmanızı koruyun [58].

```
docker version
docker compose version
docker info --format '{{.OSType}}'
```

Beklenen sonuç: docker version çıktısında Client ve Server bilgileri; Compose sürümü; son komutta linux görülür. Server bağlantı hatasında Docker Desktop'ı açın ve hazır olmasını bekleyin. windows görünürse Linux container moduna geçin. Bu kontroller geçmeden devam etmeyin.

Tablo 1. Uygulamanın temel çalışma değerleri.

Öğe	Değer
Web servisi	web; container portu 8000
Tarayıcı erişimi	http://localhost:8080
Veri servisi	db; PostgreSQL 16; port 5432
Veritabanı ve kullanıcı	dersnotlari ve dersuser
Compose projesi	hafta5
Kalıcı volume	hafta5_db-verisi

Aynı bilgisayarda birden fazla öğrenci hesabı aynı Docker Engine'i kullanıyorsa proje adı ve ana makine portu çakışabilir. Kişisel bilgisayarda verilen değerleri değiştirmeyin; ortak laboratuvar için öğretim elemanının belirlediği benzersiz değerleri kullanın.

3 Yapılandırma ve gizli bilgi dosyalarını hazırlama

Secret (gizli bilgi), burada veritabanı parolası ve uygulama anahtarıdır. Bu değerleri imaja veya Compose dosyasına yazmak yerine secrets klasöründeki dosyalardan sunarız. APP_SECRET, form doğrulamasındaki oturumları imzalayan uygulama anahtarıdır; veritabanı parolasıyla aynı değildir [53].

Aşağıdaki blok yalnız ilk hazırlıkta çalıştırılır. Var olan anahtarları değiştirmek, özellikle mevcut veritabanı parolasını tutarsız hâle getirebilir. İşlev bu nedenle var olan dosyanın üstüne yazmaz.

```
New-Item -ItemType Directory -Path .\secrets -Force | Out-Null
function New-LabSecret {
   param([string]$Path)
   if (Test-Path $Path) {
      throw "Dosya zaten var; mevcut gizli bilgiyi koruyun."
   }
   $bytes = New-Object byte[] 32
   $rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
   try { $rng.GetBytes($bytes) } finally { $rng.Dispose() }
   $value = [Convert]::ToBase64String($bytes)
   $encoding = New-Object System.Text.UTF8Encoding($false)
   $fullPath = Join-Path (Get-Location).Path $Path
   [System.IO.File]::WriteAllText($fullPath, $value, $encoding)
}
New-LabSecret "secrets\db_password.txt"
New-LabSecret "secrets\app_secret.txt"
Set-Content -Path .\.env -Value 'APP_IMAGE_TAG=1.0' -Encoding ASCII
Get-ChildItem .\secrets | Select-Object Name, Length
Get-Content .\.env
```

Beklenen sonuç: İki dosyanın her biri 44 bayttır; içerikleri ekrana yazdırılmaz. .env yalnız APP_IMAGE_TAG=1.0 içerir. PowerShell 5.1 ve 7 için gizli bilgi dosyaları UTF-8 olarak, byte order mark (bayt sırası işareti, BOM) ve satır sonu olmadan yazılır.

.env, bu örnekte Compose'un imaj etiketini yerine koyması içindir; bütün içeriği kendiliğinden container'a aktarılmaz. Paketteki .env.example, uygulamanın tanıdığı seçenekleri gösteren başvuru dosyasıdır; burada onu .env olarak kopyalamayın. Parolalar dosya tabanlı yol üzerinden verilir.

Dosyaları paylaşmayın ve ekran görüntülerine içeriklerini almayın. Yerel Compose secrets, kaynak dosyayı kendiliğinden şifrelemez. Windows hesabınızın klasör izinlerini koruyun. Aynı dosyalar container yeniden oluşturulurken kullanılmalıdır [53].

4 Dockerfile ile uygulama imajını tanımlama

Çalışma klasöründe uzantısız Dockerfile oluşturun. Dosya adının Dockerfile.txt olmadığını kontrol edin. Aşağıdaki iki komut yeni dosyayı açar; dosya zaten varsa New-Item komutunu tekrar çalıştırmadan yalnız Notepad ile açın.

```
New-Item -ItemType File -Path .\Dockerfile | Out-Null
notepad .\Dockerfile
```

Dosyaya aşağıdaki içeriğin tamamını yapıştırıp kaydedin. Bunlar PowerShell komutları değil Dockerfile yönergeleridir.

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
RUN useradd --create-home --uid 10001 appuser
COPY --chown=appuser:appuser app.py ./
COPY --chown=appuser:appuser templates/ ./templates/
COPY --chown=appuser:appuser static/ ./static/
USER appuser
EXPOSE 8000
CMD ["gunicorn", "--bind=0.0.0.0:8000", \
    "--workers=1", "--threads=2", "--timeout=30", "app:app"]
```

FROM, Python çalışma ortamını seçer. requirements.txt önce kopyalanıp bağımlılıklar kurulur; uygulama kodu sonradan kopyalanır. Böylece yalnız uygulama kodu değiştiğinde bağımlılık katmanı önbellekten kullanılabilir. Parola dosyaları kopyalanmaz [21][24].

Kurulum işlemleri tamamlandıktan sonra USER appuser ile web süreci root olmayan kullanıcıya geçer. EXPOSE, portu belgeler; tarayıcı erişimini tek başına açmaz. CMD, Gunicorn ile uygulamayı başlatır. app:app, app.py dosyasındaki Flask uygulama nesnesini gösterir. 0.0.0.0:8000, container'ın ağ arayüzlerinden bağlantı kabul etmesini sağlar [21].

Python imajı Docker'ın resmî imajıdır [57]. Gunicorn Linux container'ın içinde çalışır; Windows'a Gunicorn kurulmaz. Uygulamanın veri bağlantısı ve anahtarı oluşturma sırasında değil çalıştırılırken sağlanır.

5 Derleme bağlamını sınırlama ve imajı oluşturma

5.1 Dockerignore dosyası

```
New-Item -ItemType File -Path .\.dockerignore | Out-Null
notepad .\.dockerignore
```

Dosyaya aşağıdaki içeriği yazıp kaydedin:

```
secrets/
.env
.env.*
.git/
.venv/
venv/
__pycache__/
*.py[cod]
*.log
*.dump
*.zip
compose*.yaml
compose*.yml
README.md
```

.dockerignore, derleme bağlamı (build context) içinden gereksiz ve hassas dosyaları dışarıda tutar. Dockerfile'da açık COPY kullanılmasıyla birlikte korunma sağlar. Bu dosya Git'e eklenmeyi engellemez; Git kullanılıyorsa gizli bilgi dosyaları ayrıca .gitignore kapsamına alınır [22].

5.2 İmajı oluşturma ve inceleme

```
docker build --pull -t dersnotlari-web:1.0 .
docker image ls dersnotlari-web
docker image inspect dersnotlari-web:1.0 --format '{{.Config.User}}'
docker run --rm dersnotlari-web:1.0 python -c "import flask, psycopg; print('Kutuphaneler hazir')"
```

Komutun sonundaki nokta, mevcut klasörü derleme bağlamı yapar. --pull temel imajın güncel karşılığını kontrol eder; -t ad ve etiket verir. İlk derlemede internet bağlantısı gerekir. İndirme ve kurulum tamamlanınca dersnotlari-web:1.0 listelenir; kullanıcı appuser, son çıktı Kutuphaneler hazir olmalıdır [24].

Son komut uygulamanın tamamını başlatmaz; imajdaki kütüphaneleri doğrular ve geçici container'ı kaldırır. Veritabanı ve gizli bilgiler henüz verilmediğinden imajı tek başına varsayılan komutuyla çalıştırmayın. Derleme başarısı, uygulamanın veri servisiyle çalıştığını kanıtlamaz.

6 Compose dosyasını oluşturma

compose.yaml, iki servis, gizli bilgi dosyaları ve kalıcı veri alanını birlikte tanımlar. YAML (YAML Ain’t Markup Language, YAML bir işaretleme dili değildir) girintileri anlam taşır; sekme yerine boşluk kullanın. Dosyayı oluşturun:

```
New-Item -ItemType File -Path .\compose.yaml | Out-Null
notepad .\compose.yaml
```

Bu sayfadaki ilk parçayı ve sonraki sayfadaki devamını aynı compose.yaml dosyasına sırayla yazın. İlk parça tek başına tamamlanmış bir dosya değildir. Kod dışındaki açıklamaları dosyaya eklemeyin.

```
name: hafta5
services:
  web:
    image: dersnotlari-web:${APP_IMAGE_TAG:-1.0}
    build: .
    ports:
      - "127.0.0.1:8080:8000"
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: dersnotlari
      DB_USER: dersuser
      DB_PASSWORD_FILE: /run/secrets/db_password
      APP_SECRET_FILE: /run/secrets/app_secret
      COOKIE_SECURE: "false"
    secrets:
      - db_password
      - app_secret
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test:
        - CMD
        - python
        - -c
        - >-
          import urllib.request;
          urllib.request.urlopen(
          'http://127.0.0.1:8000/health/ready', timeout=8)
      interval: 15s
      timeout: 10s
      retries: 5
      start_period: 20s
    restart: unless-stopped
```

127.0.0.1:8080:8000, yalnız kendi Windows bilgisayarınızdan erişim sağlar. DB_HOST=db, Compose ağındaki veri servisini belirtir; web içinde localhost kullanmak veri servisini göstermez [30][32].

6 Compose dosyasının devamı

Aşağıdaki satırları önceki parçanın hemen altına ekleyin. db satırı iki boşluk girintilidir; en alttaki volumes ve secrets, services ile aynı düzeydedir. Dosyayı kaydedin.

```
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: dersnotlari
      POSTGRES_USER: dersuser
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    volumes:
      - db-verisi:/var/lib/postgresql/data
    healthcheck:
      test:
        - CMD-SHELL
        - pg_isready -h 127.0.0.1 -U dersuser -d dersnotlari
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 20s
    restart: unless-stopped
volumes:
  db-verisi:
secrets:
  db_password:
    file: ./secrets/db_password.txt
  app_secret:
    file: ./secrets/app_secret.txt
```

PostgreSQL 16 için veri alanı /var/lib/postgresql/data yoluna bağlanır. Bu yolu başka ana sürümlere aynen taşımayın. db-verisi, Docker'ın yönettiği named volume (adlandırılmış veri birimi) olur. Proje adı nedeniyle gerçek adı hafta5_db-verisi'dir [26][56].

İki servise aynı db_password dosyası verilir. POSTGRES_PASSWORD_FILE, PostgreSQL'in ilk hazırlanmasında kullanıcı parolasını belirler; DB_PASSWORD_FILE ise web uygulamasının bağlantıda kullandığı parolayı okur. Uygulama anahtarı yalnız web servisine sunulur [53][56].

POSTGRES_USER ile oluşturulan dersuser, bu resmî imajın başlangıç işlemi nedeniyle veritabanı içinde yüksek yetkilidir. Bu, eğitim kurulumunu basitleştirir; üretimde ayrı ve sınırlı bir uygulama kullanıcısı gerekir. Bu durum web sürecinin root olmadan çalışmasından farklıdır [56].

Veritabanının sağlık kontrolü bağlantı kabulünü, web kontrolü ise hazır uygulamanın veritabanına erişimini sınar. depends_on / service_healthy ilk başlatmada db hazır olmadan web'in başlamasını önler. pg_isready tek başına uygulama parolasının doğru olduğunu kanıtlamaz. restart ilkesi, yalnız unhealthy durumuna geçilmesiyle container'ı otomatik yeniden başlatmaz [33][34].

7 Yapılandırmayı doğrulama ve servisleri başlatma

Önce doğru klasörde ve gerekli dosyaların bulunduğundan emin olun. Dosya adları ve gizli bilgi dosyalarının uzunluklarını kontrol etmek içeriklerini ifşa etmez.

```
Get-Location
Get-ChildItem -Force
Get-ChildItem .\secrets | Select-Object Name, Length
docker compose config --quiet
docker compose config --services
```

Beklenen sonuç: config --quiet hata vermeden tamamlanır; servis listesi web ve db içerir. Tam compose config çıktısı imaj, port ve dosya yollarını incelemek için kullanılabilir. Bu örnekte gizli bilgi değerleri ortam değişkenlerine yazılmadığı için kaynak parolalar çıktıya eklenmez. Yapılandırmanın geçerli olması çalışma başarısını henüz göstermez [61].

```
docker compose pull db
docker compose up -d --build --wait --wait-timeout 180
docker compose ps
```

İlk komut PostgreSQL imajını çeker. up, web imajını gerekirse oluşturur, proje ağını ve volume'u hazırlar, servisleri başlatır. --wait, tanımlı sağlık kontrollerinin başarılı olmasını bekler; ilk indirme ve derleme süreleri bağlantıya bağlıdır. Beklenen sonuç iki servisin de Up ve healthy görünmesidir [35].

180 saniyede hazır olamazsa tekrar tekrar kurulum veya veri silme yapmayın. Durumu ve son günlükleri okuyun:

```
docker compose ps -a
docker compose logs --tail 60 db
docker compose logs --tail 60 web
```

Gunicorn'un başladığı ve PostgreSQL'in bağlantı kabul ettiği görülmelidir. Parola ve uygulama anahtarı değerlerini günlüklerde arayıp paylaşmayın. YAML hata mesajı varsa belirtilen satırın girintisini; SECRET ile ilgili hata varsa dosya adı ve yolu; bağlantı hatasında db durumunu kontrol edin. Ayrıntılı belirtiler Bölüm 13'tedir [59].

Bu kılavuzun bütün Compose komutlarında aynı proje adı ve compose.yaml kullanılır. Farklı bir klasöre geçerek veya -p ile adı değiştirerek başka proje başlatmayın; farklı volume oluşabilir.

8 Uygulamayı kullanma ve ilk doğrulama

8.1 Tarayıcıdan kontrol

Tarayıcıda http://localhost:8080 adresini açın. Beklenen sonuç: Ders Notları arayüzü, uygulama sürümü 1.0 ve Veri servisi Erişilebilir bilgisi. İlk açılışta kayıt yoktur; gerekli notes tablosunu uygulama oluşturur.

Başlık alanına İlk container kaydım, açıklamaya Web uygulaması ayrı PostgreSQL servisine bağlandı yazın ve Kaydı ekle düğmesine basın. Başka bir başlıkla ikinci kayıt ekleyin. En yeni kayıt üstte görünür; oluşturulma zamanı UTC, Coordinated Universal Time (Eşgüdümlü Evrensel Zaman) olarak gösterilir.

Ekran görüntüsünde sürüm, veri servisi durumu ve kayıtlar birlikte görünsün. Başlık en fazla 100, açıklama en fazla 1000 karakterdir. Formu boş göndermeyi deneyin; tarayıcı gerekli alanı belirtir. Sunucu da geçersiz girdileri kontrol eder. Gerçek kişisel veri kullanmayın.

8.2 Sağlık ve veri kontrolü

```
Invoke-RestMethod -Uri 'http://localhost:8080/health/live'
Invoke-RestMethod -Uri 'http://localhost:8080/health/ready'
docker compose exec db psql -U dersuser -d dersnotlari -c "SELECT id, title, created_at FROM notes
ORDER BY id;"
```

Beklenen sonuç: live için status=ok, ready için status=ready ve version=1.0; SQL çıktısında eklediğiniz başlıklar. Windows'a psql kurulmaz; exec komutu onu veri container'ının içinde çalıştırır. Bu yerel yönetim kontrolü web uygulamasının parolalı ağ bağlantısının yerine geçmez; kayıt ekleme o bağlantıyı zaten sınamıştır [60].

8.3 Kullanıcı yetkisini kontrol

```
docker compose exec web id
docker compose exec web python -c "import os; print(os.getuid())"
```

Beklenen sonuç: appuser ve kullanıcı kimliği 10001. Bu, web sürecinin root olarak çalışmadığını gösterir. PostgreSQL'in kendi kullanıcı/yetki düzenini değiştirmeyin. Formdaki gizli doğrulama anahtarı otomatik kullanılır; yeniden oluşturma sonrası form hatasında sayfayı yenileyin.

9 Ağ ve veri alanını inceleme

9.1 Bağlantı adreslerini doğrulama

```
docker compose port web 8000
docker network inspect hafta5_default
docker compose exec web python -c "import socket; print(socket.gethostbyname('db'))"
docker compose exec web python -c "from app import connect_db; c=connect_db();
print(c.execute('SELECT 1').fetchone()); c.close()"
```

Beklenen sonuç: ana makine eşlemesi 127.0.0.1:8080; ağda web ve db üyeleri; db adı için ağ içi bir adres; son kontrolde SELECT 1 sonucu. Ağ içi adresi ezberlemeyin veya uygulamaya sabitlemeyin; servis adını kullanın [32].

Tarayıcı 8080 üzerinden web'in 8000 portuna erişir. Web ise db:5432 üzerinden PostgreSQL'e bağlanır. Veri servisi için ports yazılmadığından 5432, ana makinede yayımlanmaz. docker compose ps çıktısındaki 5432/tcp ifadesi tek başına yayımlama değildir; HOST:PORT eşlemesi aranır [30].

9.2 Volume ve gizli bilgi yollarını inceleme

```
docker volume inspect hafta5_db-verisi
$dbContainerId = docker compose ps -q db
docker inspect $dbContainerId --format '{{json .Mounts}}'
docker compose exec web python -c "import os; print(os.path.isfile('/run/secrets/db_password'));
print(os.path.isfile('/run/secrets/app_secret'))"
```

Beklenen sonuç: volume adı hafta5_db-verisi, veri hedefi /var/lib/postgresql/data ve web tarafında iki True. Mounts içinde gizli bilgi bağları da görülebilir. Docker Desktop volume verisini Linux ortamında yönetir; görülen Linux yolu Windows Dosya Gezgini'ne yapıştırılacak normal bir klasör yolu değildir [26][53].

Uygulama kaynakları COPY ile imaja alınmıştır; bu Compose dosyasında kaynak kod klasörünü web'e bağlayan bind mount yoktur. Dosyanın Windows'ta değiştirilmesi çalışan container'ın kodunu kendiliğinden değiştirmez. Bölüm 11'de yeni imajla güncelleme yapılır.

10 Container yaşam döngüsü ve veri kalıcılığı deneyi

10.1 Durdurma ve yeniden başlatma

Önce en az iki kayıt oluşturun ve başlıklarını not edin. Çalışan container kimliklerini alın:

```
docker compose ps -q
docker compose stop
docker compose ps -a
docker compose start --wait
docker compose ps -q
```

Beklenen sonuç: stop sonrasında container'lar Exited; start sonrasında tekrar çalışır. Kimlikler aynıdır, çünkü bu işlem mevcut container'ları yeniden başlatır. Tarayıcıda kayıtlar kalır. Bu, yeniden oluşturma deneyinin yerine geçmez.

10.2 Container'ları kaldırıp yeniden oluşturma

```
docker compose ps -q
docker compose down
docker compose ps -a
docker volume inspect hafta5_db-verisi
docker compose up -d --wait --wait-timeout 180
docker compose ps -q
```

Beklenen sonuç: down sonrasında uygulama container'ları ve proje ağı kaldırılır; named volume kalır. up yeni container kimlikleri oluşturur ve aynı volume'u kullanır. Tarayıcıyı yenilediğinizde önceki kayıtlar aynı numara ve başlıklarla görünmelidir [26][35][38].

Kanıt için ilk kayıt ekranını, kaldırma ve yeniden oluşturma çıktısını, yeni kimlikleri ve sonraki kayıt ekranını alın. Bu deneyde down komutuna --volumes veya -v eklemeyin. Compose proje adı hafta5 olarak korunmalıdır.

Kayıtlar kaybolursa yeni kayıt eklemeden volume adını ve bağlama hedefini kontrol edin. Container'ın yeniden oluşturulması veri kaybını zorunlu kılmaz; doğru volume'un yeniden kullanılması gerekir. Kalıcılık yedekleme değildir.

11 Uygulama sürümünü güncelleme

Bu deneyde yalnız web uygulaması değiştirilir; veritabanı container'ı ve volume korunur. Windows'taki kaynak değişikliğinin yeni imaj oluşturulmadan çalışmaya yansımadığını gözlemleyin.

11.1 Kaynağı değiştirme

```
notepad .\app.py
```

Dosyada APP_VERSION = "1.0" satırını bulun; yalnız değeri "1.1" yapın ve UTF-8 olarak kaydedin. Python girintilerine ve tırnaklara dokunmayın. Bu aşamada tarayıcıyı yenileyin: çalışan imaj hâlâ 1.0 olduğundan sürüm değişmez.

11.2 Yeni imajı oluşturma ve kullanma

```
Set-Content -Path .\.env -Value 'APP_IMAGE_TAG=1.1' -Encoding ASCII
docker compose config --images
docker compose build web
docker image ls dersnotlari-web
docker compose up -d --no-deps --force-recreate web
docker compose ps
Invoke-RestMethod -Uri 'http://localhost:8080/health/ready'
```

Beklenen sonuç: yapılandırmada dersnotlari-web:1.1 ve postgres:16; yerel imaj listesinde 1.0 ve 1.1. Son komutta status=ready ve version=1.1 görülür. up yalnız web'i yeniden oluşturur; --no-deps, db'nin başlatma işlemini bu adımın dışında tutar. db bu komuttan önce çalışıyor olmalıdır [35].

Sağlık kontrolü henüz tamamlanmadıysa docker compose ps ve son günlükleri kontrol edin. Tarayıcıda 1.1 görünmeli, önceki kayıtlar korunmalıdır. Yeni etiketi yazmak tek başına uygulama kodunu değiştirmez; kaynak değişikliği, yeni derleme ve yeni container birlikte gerekir.

docker compose restart web kullanmak güncelleme adımının yerine geçmez. Kayıtlar değiştirilen web imajında değil, veri servisine bağlı volume içindedir. 1.0 imajı yerelde kalabilir; artık container tarafından kullanılmıyorsa ayrıca kaldırılabilir.

11.3 İsteğe bağlı imaj taraması

```
docker scout cves dersnotlari-web:1.1
```

Docker Scout erişimi ve oturum gereksinimleri ortamınıza bağlıdır; bu kontrol zorunlu yerel çalışmayı engellemez. Bulunan paket, önem düzeyi ve düzeltilmiş sürüm bilgilerini yorumlayın. Tarama sonucu tek başına güvenlik garantisi değildir [55].

12 Veri servisi kesintisini gözlemleme

Bu kısa deney, web sürecinin çalışması ile uygulamanın veri işlemlerine hazır olmasının farklı olduğunu gösterir. Verileri silmeden yalnız veri servisini durdurun:

```
docker compose stop db
docker compose ps -a
Invoke-RestMethod -Uri 'http://localhost:8080/health/live'
```

Beklenen sonuç: db durmuş; web süreci yanıt veriyorsa live status=ok döndürür. Tarayıcıyı yenileyin: veri servisine ulaşılamadığı mesajı görünür ve ekleme düğmesi devre dışıdır. ready adresi HTTP 503 döndürür; PowerShell bu yanıtı hata olarak gösterebilir. web'in sağlık durumunun unhealthy olması birkaç kontrol aralığı sürebilir.

```
Invoke-RestMethod -Uri 'http://localhost:8080/health/ready'
```

Bu adımda HTTP 503 beklenen sonuçtur. Veri servisi kesildi diye volume'u veya gizli bilgi dosyalarını silmeyin. Servisi yeniden başlatın:

```
docker compose up -d --wait --wait-timeout 180 db
Invoke-RestMethod -Uri 'http://localhost:8080/health/ready'
docker compose ps
```

Beklenen sonuç: db hazır olduktan sonra ready yeniden başarılıdır; tarayıcıda önceki kayıtlar görünür. Gerekirse sayfayı yenileyin. Web uygulaması her işlem için yeni bağlantı açtığından yeniden erişim sağlayabilir. Başarısız kayıt ekleme işlemini kendiliğinden tekrarlamaz; kullanıcı sonucu kontrol edip yeniden gönderir.

depends_on ilk başlangıç düzenini sağlar; veri servisinin bütün çalışma ömrü boyunca erişilebilir kalmasını garanti etmez. Unhealthy olmak, çalışan sürecin otomatik yeniden başlatıldığı anlamına gelmez [33][34].

Bu deneyden sonra yeniden bir kayıt ekleyip listeleyin. Hem sağlık kontrolünün hem gerçek uygulama işlevinin düzeldiğini göstermiş olursunuz.

13 Sık karşılaşılan sorunlar

Tablo 2. Belirtiye göre ilk kontrol ve düzeltme.

Belirti	Kontrol ve düzeltme
Docker sunucusuna bağlanılamıyor	Docker Desktop açık mı? docker version içinde Server bölümü var mı? Linux modu etkin mi? Ek 1 kontrollerine dönün.
app.py veya requirements.txt bulunamıyor	Get-Location ve Get-ChildItem -Force ile app.py bulunan klasörü kontrol edin. docker build komutunun sonundaki nokta gereklidir.
Dockerfile veya YAML hatası	Dosyanın .txt uzantısıyla kaydedilmediğini doğrulayın. Compose girintilerinde boşluk kullanın; iki parçanın aynı dosyada olduğundan emin olun.
8080 portu kullanımda	Ek 1'deki eski web container'ı açık olabilir. docker ps ile bulun, yalnız size ait deneme container'ını durdurun. Alternatif olarak eşlemeyi 127.0.0.1:8081:8000 yapın ve tarayıcıda 8081 kullanın.
APP_SECRET tanımlanmadı veya izin hatası	Dosya adını, Compose secrets tanımını ve APP_SECRET_FILE yolunu kontrol edin. Anahtarı tekrar üretmek yerine mevcut dosyanın erişimini düzeltin.
db healthy ama web HTTP 503	DB_HOST=db olmalı. DB_NAME ve DB_USER iki serviste uyumlu mu? Aynı db_password dosyası kullanılıyor mu? web günlüklerini kontrol edin.
Parolayı değiştirdikten sonra bağlantı bozuldu	POSTGRES_PASSWORD_FILE mevcut volume içindeki parolayı kendiliğinden değiştirmez. İlk kurulumda kullanılan dosyayı geri yükleyin; veri silerek düzeltmeye çalışmayın.
Eski sürüm görünüyor	app.py kaydedildi mi? APP_IMAGE_TAG ve yeni derleme doğru mu? web yeni imajdan yeniden oluşturuldu mu? restart yeterli değildir.
Yeniden oluşturmada kayıtlar kayıp	Proje adı hafta5 mi? hafta5_db-verisi var mı? Hedef /var/lib/postgresql/data mı? -v/--volumes kullanıldı mı?
Form doğrulanamıyor	Sayfayı yenileyin. APP_SECRET dosyası değiştiyse eski form/çerez geçersiz olabilir. Yerel HTTP için COOKIE_SECURE=false olmalıdır.
Derlemede paket indirme hatası	İnternet, kurum proxy ayarları ve Docker Desktop bağlantısını kontrol edin. Sertifika doğrulamasını kapatmak yerine ağ sorununu çözün.

Veri kaybı içeren sıfırlamayı ancak kayıtları korumaya gerek kalmadığında uygulayın. secrets dosyalarını yeniden üretmek veya down --volumes kullanmak genel sorun giderme adımı değildir.

14 Çalışmayı sonlandırma ve verileri koruma

14.1 Kayıtları koruyarak kapatma

```
docker compose down
docker volume inspect hafta5_db-verisi
```

Beklenen sonuç: container'lar ve proje ağı kaldırılır, veritabanı volume'u kalır. app.py, Dockerfile, .dockerignore, compose.yaml, .env ve secrets klasörü yerel çalışma klasöründedir; down bunları silmez. Aynı klasörde yeniden up çalıştırılarak devam edilebilir [38].

14.2 İsteğe bağlı veritabanı yedeği

Veriler gerekiyorsa aşağıdaki yedekleme, servisleri kapatmadan önce yapılır. PowerShell'in metin yönlendirmesiyle ikili yedek bozulmasını önlemek için önce container içinde dosya oluşturulur, ardından docker cp ile alınır. Bu küçük eğitim veritabanına yöneliktir; kurum yedekleme tasarımının yerini tutmaz.

```
docker compose exec db pg_dump -U dersuser -d dersnotlari -Fc -f /tmp/dersnotlari.dump
$dbContainerId = docker compose ps -q db
docker cp "${dbContainerId}:/tmp/dersnotlari.dump" .\dersnotlari.dump
Get-Item .\dersnotlari.dump | Select-Object Name, Length
```

Beklenen sonuç: boyutu sıfır olmayan yerel dersnotlari.dump dosyası. Geri yükleme gerektiğinde çalışan db container'ına kopyalanıp pg_restore ile alınabilir; aynı adlı mevcut kayıtların üstüne yüklemeyin. Bu dersin zorunlu teslimi için yedek dosyası istenmez.

14.3 Verileri tamamen kaldırma

Aşağıdaki işlem geri alınamaz biçimde bu projenin veritabanı volume'unu kaldırır. Yalnız kanıtları aldıktan ve veriye ihtiyaç kalmadığından emin olduktan sonra uygulayın.

```
docker compose down --volumes
docker volume ls --filter name=hafta5
```

Beklenen sonuç: hafta5_db-verisi artık listede yoktur. Yeniden up, boş bir volume ve boş veritabanı oluşturur. Uygulama kaynakları ve gizli bilgi dosyaları korunur. Yerel imajları da kaldırmak isterseniz, container'lar kaldırıldıktan sonra docker image rm dersnotlari-web:1.0 dersnotlari-web:1.1 kullanın. Genel system prune komutlarını çalıştırmayın [38].

15 Teslim ve sonraki haftaya hazırlık

Ders notunun 10.4 bölümüne uygun kısa bir rapor teslim edin. Hazır kaynak kodları, Dockerfile, .dockerignore ve Compose dosyalarının ayrıca teslim edilmesi zorunlu değildir. Sorunların incelenmesi için gerektiğinde istenebilir.

- İmaj etiketi ve çalışan iki servisi gösteren çıktı.
- Sürüm 1.0 ekranı ve eklediğiniz test kayıtları.
- Container'lar kaldırılıp yeniden oluşturulmadan önce ve sonraki kimlikler; aynı kayıtların korunduğu ekran.
- Sürüm 1.1 ekranı; güncelleme sonrası önceki kayıtların hâlâ bulunduğunun kanıtı.
- Çalışma kaynaklarının sonlandırıldığına ilişkin çıktı; volume'u koruduysanız veya sildiyseniz bunun kısa açıklaması.

Her ekran görüntüsünün altına neyi doğruladığını bir veya iki cümleyle yazın. Parolalar veya uygulama anahtarı görünmemelidir. Veritabanı başlıkları kişisel bilgi içermesin.

Kısa açıklama soruları

- Tarayıcı hangi ana makine portuna bağlanır? Web hangi container portunda dinler? Web, veri servisine hangi adresle bağlanır?
- Kayıtlar hangi volume'da saklanır? Container'lar yeniden oluşturulduğunda neden korunur? Volume silinse ne olur?
- Sürüm değişikliğinin görünmesi için hangi kaynak, imaj ve container işlemlerini yaptınız? Restart neden yeterli değildi?

Çalışma klasörünü koruma

Kaynakları ve oluşturduğunuz container dosyalarını sonraki haftanın Kubernetes uygulamasında kullanmak üzere saklayın. Çalışan container'ları açık tutmanız gerekmez. Gizli bilgileri paylaşılabilir kaynaklardan ayrı koruyun. Bulut çalışmasına geçerken yerelde doğruladığınız son imaj sürümünü kullanın; registry'ye imaj gönderilmesi volume verilerini taşımaz.

Kılavuzun teknik sınırları

Bu uygulama eğitim amaçlıdır; üyelik ve kimlik doğrulama içermez. Windows'ta yerel HTTP erişimi ve Linux container'ları hedeflenir. Python ve PostgreSQL ana sürümleri bu kılavuzda sabittir; yeni ana sürüme geçiş ayrı bir uyumluluk ve veri taşıma çalışması gerektirir. Bağımlılık güncellemelerinden sonra uygulama işlevleri ve sağlık kontrolleri yeniden doğrulanmalıdır.

16 Kaynaklar

Hafta 5 ders notunda kullanılan kaynak numaraları korunmuştur. Bu listede yalnız Ek 2'de başvurulan kaynaklar yer alır; yeni kaynaklar [56] ile devam eder. Ek 1, ortam hazırlığı için ders içi başvuru dokümanıdır. Tablo numaraları bu bağımsız ek içinde 1'den başlar.

[21] Dockerfile reference.
https://docs.docker.com/reference/dockerfile/

[22] Build context.
https://docs.docker.com/build/concepts/context/

[24] Building best practices.
https://docs.docker.com/build/building/best-practices/

[26] Volumes.
https://docs.docker.com/engine/storage/volumes/

[30] Port publishing and mapping.
https://docs.docker.com/engine/network/port-publishing/

[32] Networking in Compose.
https://docs.docker.com/compose/how-tos/networking/

[33] Define services in Docker Compose.
https://docs.docker.com/reference/compose-file/services/

[34] Control startup and shutdown order in Compose.
https://docs.docker.com/compose/how-tos/startup-order/

[35] docker compose up.
https://docs.docker.com/reference/cli/docker/compose/up/

[38] docker compose down.
https://docs.docker.com/reference/cli/docker/compose/down/

[53] Manage secrets securely in Docker Compose.
https://docs.docker.com/compose/how-tos/use-secrets/

[55] Docker Scout.
https://docs.docker.com/scout/

[56] Postgres Docker Official Image.
https://hub.docker.com/_/postgres

[57] Python Docker Official Image.
https://hub.docker.com/_/python

[58] Microsoft PowerShell Expand Archive.
https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.archive/expand-archive

[59] docker compose logs.
https://docs.docker.com/reference/cli/docker/compose/logs/

[60] docker compose exec.
https://docs.docker.com/reference/cli/docker/compose/exec/

[61] Docker Compose yapılandırma doğrulaması.
https://docs.docker.com/reference/cli/docker/compose/config/